Initialization of Convolutional Neural Networks by Gabor Filters

Gökhan Özbulak, *Student, ITU*, and Hazım Kemal Ekenel, *Assoc. Prof., ITU*

***Abstract*—In the field of the Transfer Learning (TL), for a given classification task, the learning from source domain into target domain is achieved by training a pre-trained network with data in target domain. Therefore, a pre-trained network is a pre-requisite for transferring the distribution of source domain into target domain in order to construct a classification model for target task. In this study, by contradicting this fact, the need of pre-trained networks for transfer learning is eliminated by leveraging Gabor-based filters. A sample Convolutional Neural Network (CNN) is constructed by initializing the first layer, which represents the low-level features (edges etc.) after training, with pre-determined Gabor filters that have similar low-level characteristics. Experimental results on Mnist, Cifar-10 and Cifar-100 datasets show that Gabor filter based initialization has similar effect of training with pre-trained CNNs and may be utilized where a pre-trained network is not available for transfer learning.**

***Index Terms*—domain adaptation, Gabor filter, transfer learning.**

I. Introduction

Transfer Learning (TL) is an approach in machine learning that aims to leverage the knowledge obtained through learning in one domain in order to realize a learning process in a different domain. In TL, a learning model trained on source data distribution may be transferred into another learning model that will be trained on target data distribution. By utilizing from one of the TL strategies, a general-purpose recognition model may be a source domain model in order to transfer that model into specific purpose recognition model with target data distribution that wasn't seen by source model before. For instance, a generic object recognition model may be transferred into a face recognition model, or, a face recognition model may be transferred into a facial age/gender recognition model with applying a proper TL strategy.

The steps of transfer learning have similar characteristics as in traditional machine learning approaches. It requires a model definition and a bulk of train data to learn the target recognition task. Differently from traditional machine learning techniques, in TL, the model is based on a pre-trained/frozen model rather than a definition. Depending on TL strategy, the pre-trained model may be re-trained with train data or may be used as a feature extractor in order to represent the input with source domain knowledge and to train it with another classification method such as Support Vector Machines (SVMs) or Boost Classifiers.

Recent advancements in deep learning have increased attention through transfer learning studies. Deep learning is another approach in machine learning, which imitates human actions performed by brain, with human-close or higher performance. The deep learning studies make tens of successful recognition models available to leverage them for specific tasks. The availability of such pre-trained models boosted transfer learning research as well. Especially, in domains where train/labeled data is limited, transfer learning is utilized by training a pre-trained deep network with limited data in order to obtain recognition models with human-level accuracy.

Convolutional Neural Network (CNN) is one of the widely used deep networks in transfer learning process. A CNN is basically a network topology consisting of the stack of convolutional and connected neural layers. The task of learning feature representation occurs in convolutional layers while the task of classification is performed in connected neural layers. According to chosen strategy, transfer learning sometimes affects convolutional layers or it sometimes combines feature extraction capability of CNN with other type of classifiers as mentioned above. Affecting convolutional layers in TL occurs as updating the parameters of the convolution filters in these layers during train phase. The resulting state of the parameters changes depending on the level of the layers. The convolution filters of early layers represent low-level features such as corners, blobs and edges whereas the filters in the late convolutional layers represent high-level features such as task-specific features. A feature representation from early layers of the convolutional layers can be examined in Fig. 1 [13]. In case of applying transfer learning for a target classification task, using a pre-trained CNN model allows filter parameters learnt in the convolutional layers of that CNN model to be leveraged for target task. In other words, a prior knowledge obtained during training of pre-trained model for a source task is transferred into target task over the parameters of the convolution filters of pre-trained CNN.

In this study, the need of using pre-trained CNN model for transfer learning is eliminated by directly generating a Gabor-based convolutional filter bank as low-level features and initializing first convolutional layer with these Gabor filters in the filter bank. The Gabor filters are generated based on the number of convolution filter defined for the CNN in interest and injected very first layer of the network during initialization. A sample CNN model inspired from [1] is constructed for the experiments performed on Mnist [2], Cifar-10 [3] and Cifar-100 [3] datasets. Experimental results show that proposed Gabor-based initialization schema has similar effect with pre-trained models in transfer learning and may be preferred because of its easy integration. Briefly, the contribution of this paper is two-folds: 1) Dependency to a pre-trained network is eliminated, and 2) an easy and fast transfer learning schema through layer initialization is proposed.

The rest of the paper is organized as follows: In Section II, the studies in the literature about transfer learning strategies are reported. In Section III, proposed method is explained in detail. Experimental analysis is reported in Section IV and the paper is concluded with future directions in Section V.

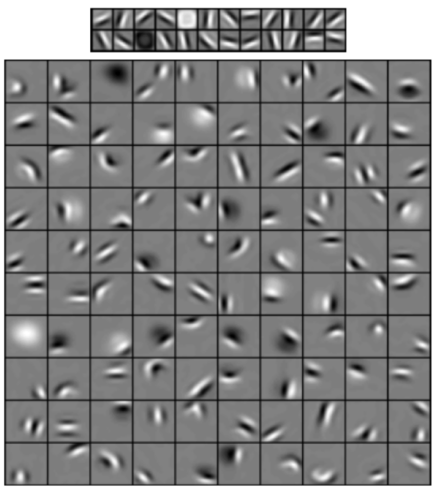


Fig. 1. The first (top) and the second (bottom) layers of a pre-trained CNN model (Courtesy of [13]).

II. Related Work

Although transfer learning is applied in machine learning for years, the advancements in recent years have more impact in this area. Especially, the human-level performance in deep learning applications made it possible to combine deep network models with transfer learning strategies in order to transfer an outperformed deep network in source domain into the target domain in interest.

A recent study [4] shows the strength of the transfer learning on various computer vision tasks such as image classification, scene recognition and image retrieval by extracting feature vector from OverFeat network [5], which is a publicly available CNN model, and combining it with Support Vector Machine (SVM) classifier. In [6], transferability of deep neural networks is investigated by focusing on transition from generality of early network layers to specificity of late network layers. In this study, it's stated that the features of first layer of a network is similar to Gabor filters and it's claimed that initialization of a network with the features of pre-trained networks increases the generalization capability of the target model. In the literature, there are many applications that exploit the power of transfer learning for various problems such as visual tracking [7], facial age and gender classification [8] and ultrasound transducer localization [9]. One recent work [10] focuses on modelling the statistical information of the good convolutional filters with Gaussian Mixture Model (GMM) and eliminates the need for pre-trained networks similarly as in this study. A detailed study about transfer learning research can be examined in [11].

III. Proposed Method

In this section, first, the steps of the Gabor filter bank generation are reported (Section III.A) and then sample network topology used for the experiments is explained (Section III.B). Finally, the injection method of prepared Gabor-based convolution filters into sample network is presented (Section III.C).

A. Gabor-Based Filter Bank

A Gabor filter is a hand-crafted, linear convolution filter used for various computer vision applications such as edge detection and texture analysis. Its bandpass characteristic allows the specific patterns (edge, blob etc.) in the image to be exposed and the rest to be suppressed. In general, Gabor filters are designed as in a filter bank to respond the edges and textures of varying frequencies and orientations. Such a constructed Gabor filter bank exhibits a vision system similar to human visual perception. Therefore, Gabor filters are successful for classification tasks which are usual for human vision.

In spatial domain, a Gabor filter is constructed from a complex Gabor function that has two components: a Gaussian-shaped function and a sinusoidal planar wave function. The basic formula of such Gabor function is as follows:

$$g(x,y) = w_r(x,y)s(x,y) \tag{1}$$

where $w_r(x,y)$ and $s(x,y)$ are the Gaussian and sinusoidal functions respectively. In order to convert the Gabor function above into a 2-D convolution filter, the function is re-formulated as below:

$$g(x,y,\sigma,\theta,\lambda,\gamma,\psi) = e^{\left(-\frac{x^2+\gamma^2 y^2}{2\sigma^2}\right)} e^{\left(i\left(2\pi\frac{x}{\lambda}+\psi\right)\right)} \tag{2}$$

where σ is the standard deviation of the Gaussian function, θ is the filter orientation, λ is the wavelength of the sinusoidal factor, γ is the spatial aspect factor and ψ is the phase offset.

In proposed method, a Gabor filter bank is constructed for the first convolution layer of the CNN model. Therefore, the total number of Gabor filter to be created in the bank depends on the number of channels/output in the first convolution layer of the CNN model. For the sample CNN model used in this study, a filter bank with 96 Gabor filters is constructed because there are 96 channels/output in first convolution layer.

The setting of the kernel parameters for constructed Gabor filters is adjusted uniformly with pre-defined intervals of those parameters. The interval of sigma, theta, lambda, gamma and psi is listed in Table I. For the sample CNN model used in this study, the interval of each parameter is divided into 96 uniformly and each Gabor filter is constructed with these parameters. A visual representation of some constructed Gabor filters for the sample CNN model can be examined in Fig. 2.

TABLE I
GABOR PARAMETER SETTING

Parameter	Name	Interval
σ	sigma	[2,21]
θ	theta	[0,360]
λ	lambda	[8,100]
γ	gamma	[0,300]
ψ	psi	[0,360]

B. *Sample CNN Model*

Sample CNN topology is inspired from [1]. As seen in Table II, the first convolution layer has 96 channels with 5x5 filters and each channel is Rectified Linear Unit (ReLU) activated. The second convolution layer has 1x1 filters to generate 96 channels and these channels are again activated by ReLU. The resulting channels are then pooled by max-pooling filter with a size of 3x3 and a stride of 2. Max-pooled channels are then fed into a convolution layer that has 5x5 filters to generate 192 channels. After these channels are activated by ReLU, they are convolved in next convolution layer that has 192 channels with 1x1 filters. The channels in this layer are again activated by ReLU and then pooled with max-pooling filter, which has a size of 3x3 and a stride of 2, as before. This is followed by a convolution layer with a channel of 192 and a filter size of 3x3. After ReLU activation, the channel is fed into another convolution layer with a channel of 192 and a filter size of 1x1. The resulting channels are activated by ReLU and then convolved in another convolution layer with a channel of 10 and a filter size of 1x1. The output is pooled by average-pooling filter with a size of 6x6 and then fed into fully connected neural layer for classification. The softmax is preferred for distributing the classification labels over the network output. In convolutional layers, each convolution filter (expect first layer, see Section III.C) is initialized from Glorot (Xavier) uniform distribution [12]. Biases are initialized with zero in all layers.

Stochastic Gradient Descent (SGD) is used as optimization algorithm for the sample CNN model. A learning rate of 0.01, a momentum of 0.9, a learning rate decay of 0.0005 and Nesterov momentum are applied during gradient descent process. Categorical cross entropy, which is formulated below, is used as loss function during training phase:

$$H(p,q) = -\sum_x p(x) \log q(x) \tag{3}$$

where $p(x)$ refers to true distribution and $q(x)$ refers to predicted distribution.

C. *CNN Initialization*

Gabor filters correspond to low-level features in a pre-trained deep network as highlighted in [6]. Therefore, generated Gabor-based convolution filters are injected into first layer of the sample CNN model during training of the target classification task. This injection is achieved by the initialization of the first convolution layer of the sample CNN model. A convolution layer has a number of convolutional filters/kernels and they must be properly initialized before starting training. When training a CNN with a pre-trained model for transfer learning, the kernel parameters of CNN model to be trained is transferred from the layers of the pre-trained model and these parameters are updated through optimization algorithm (Stochastic Gradient Descent etc.) and back propagated through loss function (Categorical Cross Entropy etc.). In proposed model, instead of using the parameters of pre-trained model as in usual transfer learning, Gabor-based convolution filters are generated and the convolution filters of CNN model to be trained are initialized with these Gabor filters. Differently from pre-trained model transfer, here, only the filters in the first convolution layer is initialized with Gabor-based filters and the filters of the remaining convolution layers are initialized from Glorot (Xavier) uniform distribution (or any other random initialization schema) as mentioned before.

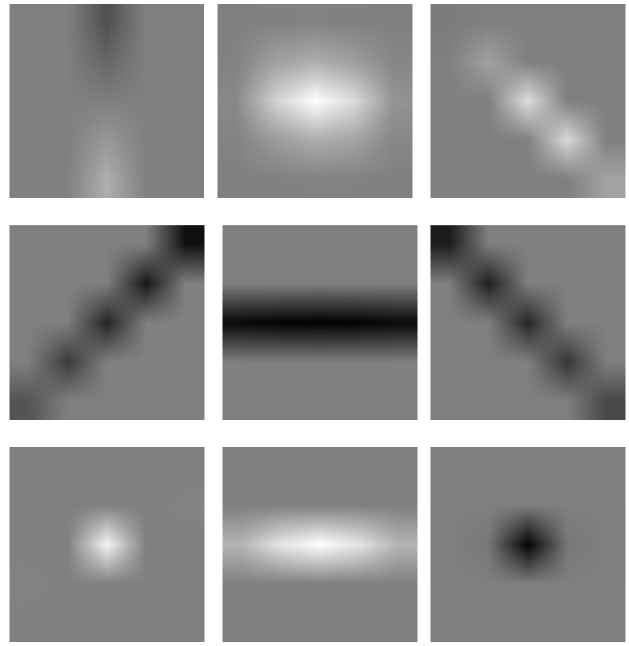


Fig. 2. Some Gabor filter samples from generated Gabor filter bank (20 times enlarged version).

The first layer of the sample CNN model has 96 channels. It means that there are 96 convolutional operations for the input and this should be handled with 96 convolution filters. Therefore, for the Gabor filter bank, 96 Gabor-based convolution filters are uniformly generated in pre-defined parameter intervals of Gabor filtering (see Table I). Briefly, the number of Gabor filters in the bank depends on the channel number defined for the first convolution layer of the CNN model in interest.

IV. EXPERIMENTAL ANALYSIS

In this section, the detail about the datasets used during experiments are explained (Section IV.A) and the experimental results of proposed Gabor-based transfer learning strategy are analyzed (Section IV.B).

TABLE II
SAMPLE CNN TOPOLOGY

Layer	#Channels (Output)	Parameters
Convolution	96	size=5x5, stride=1
ReLU		
Convolution	96	size=1x1, stride=1
ReLU		
Pooling		size=3x3, stride=2, max
Convolution	192	size=5x5, stride=1
ReLU		
Convolution	192	size=1x1, stride=1
ReLU		
Pooling		size=3x3, stride=2, max
Convolution	192	size=3x3, stride=1
ReLU		
Convolution	192	size=1x1, stride=1
ReLU		
Convolution	10	size=1x1, stride=1
ReLU		
Pooling		size=6x6, average
Softmax	10 or 100	

A. Datasets

The transferability of the Gabor initialized sample CNN model is tested on three different datasets: Mnist, Cifar-10 and Cifar-100.

Mnist dataset [2] contains the handwritten ten digits and it's one of the widely-used benchmarks for deep learning studies. It contains a training set of 60000 images and a test set of 10000 images. Each image is gray-scale and has a size of 28x28. There are ten labels representing ten digits from 0 to 9.

Cifar-10 dataset [3] is an object dataset and it's another widely used benchmark like Mnist. It has 50000 train images and 10000 test images. Each image is colored and has a size of 32x32. The dataset contains images in 10 classes ranging from birds to trucks.

Cifar-100 dataset [3] is very similar to Cifar-10 dataset. It consists of 50000 train images and 10000 test images. Each image is 32x32 colored as in Cifar-10. Cifar-100 differs from Cifar-10 for the number of the class, it has 100 classes rather than 10 classes.

B. Experimental Results

All experiments are performed with a batch size of 32 in 10 epochs. In case of train accuracy, the trained model is tested with train split of the dataset. and in case of test accuracy, the trained model is tested with test split of the dataset.

Mnist. On train set, a significant gap in the accuracy of Gabor initialized model is observed as seen in Fig. 3. There is almost 25% improvement in the accuracy. Later stages of the learning exhibit similar behavior for two models in terms of accuracy. The reason of not seeing a significant gap between two models in further epochs is that the accuracy is already very high, it's almost 100%. As seen in Fig. 3, after an improvement in accuracy about 2% in the first epoch, both model exhibits similar performance on test set because of same reason as mentioned in train set case.

In Table III, the performance of both Gabor and Glorot initialized models is listed in detail.

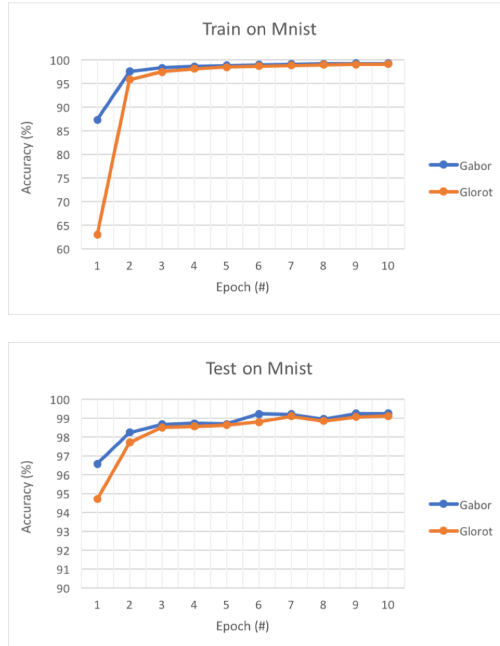


Fig. 3. The performance of Gabor initialized and Glorot initialized models on Mnist.

Cifar-10. As seen in Fig. 4, Gabor initialized model has about 8% improvement in early stages of the training. Differently from Mnist case, this gap is almost preserved in later stages of training as well. On test set, Gabor initialized model has a significant improvement of 10% against Glorot initialized model and this gap continues in later phase of training. In the end, both model almost converges with an accuracy difference of 1.5%.

The performance of both Gabor and Glorot initialized models is listed in Table IV.

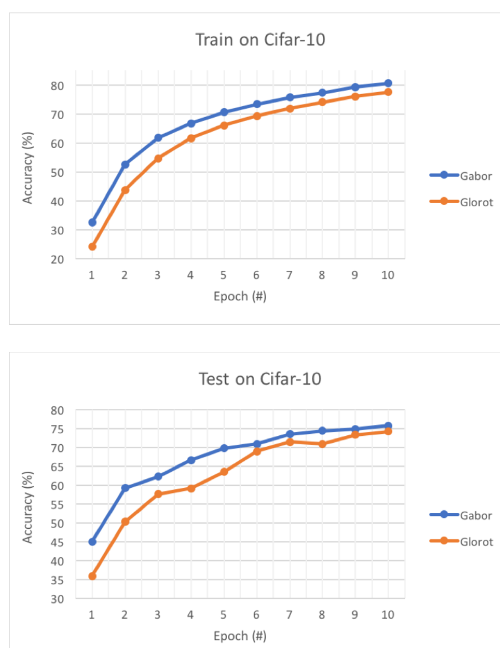


Fig. 4. The performance of Gabor initialized and Glorot initialized models on Cifar-10.

TABLE III
PERFORMANCE ON MNIST

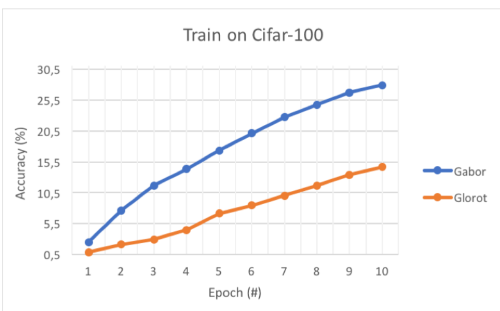
	Train		Test	
Epoch	Gabor	Glorot	Gabor	Glorot
1	87,36	63,01	96,6	94,71
2	97,56	95,85	98,26	97,71
3	98,35	97,55	98,69	98,53
4	98,7	98,18	98,74	98,58
5	98,89	98,54	98,7	98,65
6	99,03	98,71	99,23	98,8
7	99,15	98,9	99,22	99,11
8	99,21	99,03	98,97	98,86
9	99,31	99,13	99,25	99,07
10	99,33	99,2	99,25	99,13

TABLE IV
PERFORMANCE ON CIFAR-10

	Train		Test	
Epoch	Gabor	Glorot	Gabor	Glorot
1	32,44	24,05	45	35,95
2	52,5	43,73	59,21	50,28
3	61,77	54,59	62,24	57,59
4	66,72	61,57	66,64	59,1
5	70,6	65,99	69,8	63,54
6	73,36	69,26	70,9	68,93
7	75,62	71,87	73,54	71,51
8	77,33	74,08	74,4	70,92
9	79,26	76,01	74,85	73,34
10	80,5	77,51	75,73	74,22

Cifar-100. On both of train and test sets, Gabor initialized model has better accuracy in earlier of training. However, differently than the experiments performed on Mnist and Cifar-10, here, the performance gap increases with the number of epoch as seen in Fig. 5. In the end, there is an improvement of 13% on both sets by Gabor initialized model.

In Table V, the performance of both Gabor and Glorot initialized models is listed in detail.



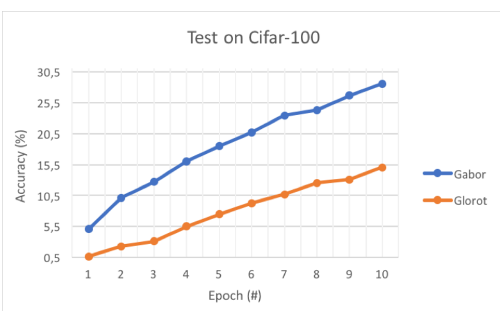


Fig. 5. The performance of Gabor initialized and Glorot initialized models on Cifar-100.

Briefly, for all experiments, it's observed that there is an early/sudden accuracy improvement by Gabor initialized model. And also, the most of the case, proposed method has better performance on both train and test sets during whole train phase. Early/sudden accuracy improvement and the increase in resulting model are two main characteristics of transfer learning applications. And, observing such characteristics in proposed method supports the claim of this study and verifies the transfer capability of Gabor filters.

TABLE V
PERFORMANCE ON CIFAR-100

	Train		Test	
Epoch	Gabor	Glorot	Gabor	Glorot
1	2,51	0,87	5,06	0,68
2	7,66	2,13	10,16	2,31
3	11,68	2,92	12,72	3,12
4	14,32	4,48	16,03	5,5
5	17,34	7,15	18,49	7,48
6	20,12	8,47	20,71	9,28
7	22,75	10,04	23,45	10,66
8	24,75	11,66	24,33	12,52
9	26,71	13,39	26,64	13,11
10	27,89	14,68	28,55	15,06

V. CONCLUSIONS AND FUTURE WORK

In this study, the need of pre-trained networks for transfer learning is eliminated by directly initializing the first layer of a CNN model with Gabor-based convolution filters. For this, a Gabor filter bank with convolution filters is generated and it's injected into the first convolution layer of a CNN in order to transfer the Gabor filter parameters into layer filters. Thus, an easy and fast way of transfer learning is proposed without any dependency to a pre-trained deep network. Experiments are performed with a Gabor initialized sample CNN model on three well-known datasets; Mnist, Cifar-10 and Cifar-100, and it's shown that a Gabor initialized CNN model indeed exhibits a model transfer characteristic and may be used as a transfer learning strategy when a proper pre-trained model doesn't exist for the target domain in which the classification task is defined. This study is planned to be extended with different network topologies (well-known shallow and deep networks etc.) and with different type of datasets to test the generalization capability of proposed method.

REFERENCES

[1] Springenberg, J. T., Dosovitskiy, A., Brox, T., & Riedmiller, M. (2014). Striving for simplicity: The all convolutional net. arXiv preprint arXiv:1412.6806.

[2] LeCun, Y. (1998). The MNIST database of handwritten digits. http://yann. lecun. com/exdb/mnist/.

[3] Krizhevsky, A., & Hinton, G. (2009). Learning multiple layers of features from tiny images.

[4] Sharif Razavian, A., Azizpour, H., Sullivan, J., & Carlsson, S. (2014). CNN features off-the-shelf: an astounding baseline for recognition. In Proceedings of the IEEE conference on computer vision and pattern recognition workshops (pp. 806-813).

[5] Sermanet, P., Eigen, D., Zhang, X., Mathieu, M., Fergus, R., & LeCun, Y. (2013). Overfeat: Integrated recognition, localization and detection using convolutional networks. arXiv preprint arXiv:1312.6229.

[6] Yosinski, J., Clune, J., Bengio, Y., & Lipson, H. (2014). How transferable are features in deep neural networks?. In Advances in neural information processing systems (pp. 3320-3328).

[7] Gao, J., Ling, H., Hu, W., & Xing, J. (2014, September). Transfer learning based visual tracking with gaussian processes regression. In European Conference on Computer Vision (pp. 188-203). Springer, Cham.

[8] Ozbulak, G., Aytar, Y., & Ekenel, H. K. (2016, September). How Transferable Are CNN-Based Features for Age and Gender Classification? In Biometrics Special Interest Group (BIOSIG), 2016 International Conference of the (pp. 1-6). IEEE.

[9] Heimann, T., Mountney, P., John, M., & Ionasec, R. (2014). Real-time ultrasound transducer localization in fluoroscopy images by transfer learning from synthetic training data. Medical image analysis, 18(8), 1320-1328.

[10] Aygün, M., Aytar, Y., & Ekenel, H. K. (2017). Exploiting Convolution Filter Patterns for Transfer Learning. arXiv preprint arXiv:1708.06973.

[11] Pan, S. J., & Yang, Q. (2010). A survey on transfer learning. IEEE Transactions on knowledge and data engineering, 22(10), 1345-1359.

[12] Glorot, X., & Bengio, Y. (2010, March). Understanding the difficulty of training deep feedforward neural networks. In Proceedings of the Thirteenth International Conference on Artificial Intelligence and Statistics (pp. 249-256).

[13] Lee, H., Grosse, R., Ranganath, R., & Ng, A. Y. (2009, June). Convolutional deep belief networks for scalable unsupervised learning of hierarchical representations. In *Proceedings of the 26th annual international conference on machine learning* (pp. 609-616). ACM.